

一种块级连续数据保护系统的快速恢复方法

李 琥, 谭毓安, 李元章

(北京理工大学 计算机学院, 北京 100081)

摘 要: 针对传统连续数据保护技术需要遍历连续数据保护(CDP)日志以提供任意时间点磁盘镜像的问题,提出了一种新的快速恢复技术,即基于后向连接的按需快速恢复技术. 在讨论了该快速恢复技术使用的基本数据结构,并引进了在基本数据结构中加入指针构成的后向连接的基础上,给出并分析了利用后向连接实现任意时间点快速恢复的相关算法. 通过在实际工作负载下的模拟实验,说明了在恢复时间点距离当前时间点较近的情况下,使用快速恢复方法能够以可接受的响应时间为代价完成快速恢复.

关键词: 连续数据保护; 按需恢复; 临时磁盘映像

中图分类号: TP 301

文献标志码: A

文章编号: 1001-0645(2011)06-0679-06

A Rapid Recovery Method for Block-Level CDP Systems

LI Xiao, TAN Yu-an, LI Yuan-zhang

(School of Computer Science and Technology, Beijing Institute of Technology, Beijing 100081, China)

Abstract: In this paper, we propose a new rapid recovery method for continuous data protection (CDP) system to solve the problem in traditional CDP system that it takes large amount of time to recover to any point in time. The proposed method could be called by the technique of back-link based recovery on demand. After discussing the data structure of new method and introducing the concept of back-link based on the pointers in the data structure, we have developed and constructed the new rapid recovery algorithms. The test has been implemented by using typical workload trace data. Experimental result shows that, when the recovery point is near the current time point, the rapid recovery method can present the temporary disk image with little loss of response time.

Key words: continuous data protection(CDP); recovery on demand; temporary disk image

随着信息技术的不断发展,数据信息已在越来越多的企业和组织中扮演着至关重要的角色.但是,数据信息同时也正面临着各种潜在的风险,例如人为或者技术上的错误,自然灾害,甚至恐怖袭击.根据统计数据显示,硬件故障成为最大风险来源,在所有数据损失中,有 44% 源自硬件故障.用户误操作导致的数据损失占 32%,软件错误占 14%,而计算机病毒和自然灾害导致的数据损失分别占总数的 7% 和 3%^[1].

为了保护重要的数据信息,可采用数据备份技

术,在数据不可用时能从备份介质中恢复数据.但是在恢复数据时,两次备份之间的数据会丢失.对于一些关键的信息系统,所能容忍的数据丢失量即恢复点目标(recovery point object, RPO)要尽可能地低,需要应用连续数据保护(continuous data protection, CDP)技术^[2-4].

CDP 将更改过的所有数据按时间顺序保存下来,每次写操作都会生成带有时间戳的数据块版本,在恢复数据时能够获取任意一个时间点的数据状态.块级 CDP 将一个磁盘即逻辑单元(logical unit,

收稿日期: 2010-09-29

作者简介: 李琥(1982—),男,博士生, E-mail: tiger-9@263.net; 谭毓安(1972—)男,教授,博士生导师, E-mail: victortan@yeah.net.

通信作者: 李元章(1978—),男,在职博士生,讲师, E-mail: popular@bit.edu.cn.

LUN)划分为固定大小的数据块,以数据块为单位记录数据变化,特点是与应用的耦合比较低,性能和效率优于文件级 CDP^[5-7].

当需要恢复数据到某个指定的时间点时,CDP 系统可以从开始时间点遍历所有已经记录的修改过的数据块,一直到指定的时间点,以恢复该时间点的数据状态,即获得该时间点的磁盘的每一个逻辑块地址(logical block address, LBA)与 CDP 所记录的数据块之间的映射关系。

为了缩短遍历时间,通常需要在 CDP 系统中定期产生数据快照,将某个时刻的映射关系存储起来。要恢复快照时刻的数据时,就可以直接获得映射关系,而无需再遍历 CDP 日志。要恢复到非快照时间点的数据时,需要从该时间点的上一次快照开始,遍历 CDP 日志,以获得该时间点的映射关系。该时间点与上次快照之间的时间间隔越长,数据变化量相应增加,恢复所需的时间就越长。

但是,在实际应用中,由于储存空间的限制,不可能以较短的时间间隔储存大量快照,并且系统恢复通常发生在系统出现故障后不久,因此恢复时间点距离当前时间点一般较近。另外,用户通常只需要从 CDP 系统提供的磁盘映像中恢复少量文件或查看个别文件的内容,即只需要读取磁盘映像中的一小部分数据块,但在传统的 CDP 系统中,要恢复过去的任意时间点,必须等待 CDP 系统遍历日志区以获得所有数据块的映射表之后,才能使用磁盘映像^[8-11]。

针对以上问题,作者提出了一种在块级 CDP 中对任意时间点数据的快速恢复方法。在生成某一时间点的磁盘映像时,CDP 系统不需要获取磁盘全部数据块的映射关系。当计算机读取磁盘映像的某一个数据块时,CDP 系统从日志中获取该数据的映射关系,从 CDP 数据区中获取数据块内容。该方法实现了磁盘数据块的按需恢复,能够在较短的时间内为用户提供任意时间点的磁盘映像。

1 快速恢复方法的数据结构

CDP 系统的快速恢复方法通过在 CDP 元数据结构中增加额外的指针字段从而引入后向链接实现。因此,首先需要说明基本 CDP 系统使用的数据结构。

在具有最基本功能的传统 CDP 系统中,对磁盘数据块的每次写操作,CDP 系统中都有一个记录该

操作的 CDP 元数据结构。CDP 元数据包含数据块的块号和时间戳。当 CDP 系统捕获到对磁盘数据块的 1 次写操作时,生成 1 个元数据结构。CDP 系统在日志数据区中记录数据块内容,在日志元数据区中记录数据块元数据。同时 CDP 系统还需要实时维护 1 个映射表,表中每一项记录 1 个磁盘数据块最后 1 次写入操作元数据在 CDP 日志中的位置,并且使用 1 个特殊值表示从开始记录 CDP 日志以来尚未修改的数据块。

当 CDP 系统需要保存当前时刻的快照时,将系统当前时刻的映射表内容作为快照元数据保存在磁盘上。当需要恢复已经保存在磁盘上的快照时,从快照元数据中获取快照时刻的映射表内容,再根据映射表中指定的每个数据块最后一次修改在 CDP 日志中的位置,即可恢复当时的磁盘数据内容。

当需要恢复快照之外的其他时间点数据时,首先恢复到该时间点之前的一次快照,获得快照时刻的映射关系,再遍历快照时刻到该时间点之间的 CDP 元数据,根据元数据内容更新映射关系。

快速恢复方法通过在上述过程中的 CDP 元数据中增加指针字段,实现对任意时间点的快速恢复。通过引进指针字段,每个 CDP 元数据获得了后向连接属性,对每一个磁盘数据块的修改操作,在 CDP 日志中形成了一个单向链表,为任意时间点的按需恢复提供了基础。因此,下文中也将 CDP 元数据称为链表节点。

具体地,CDP 任意时间点快速恢复方法使用了以下数据结构。

1.1 日志

CDP 日志的数据区和元数据区中分别保存数据块、CDP 元数据(即链表节点)。按文件系统或上层应用的最小访问粒度确定数据块的大小(例如每块大小为 4 kB),将整个磁盘划分 N 个数据块,块地址的范围为 $0 \sim N-1$ 。

CDP 日志数据区记录每一次数据块写操作的数据内容,由 M 个块组成,块的编号为 $0 \leq k \leq M-1$ 。

1.2 链表节点

每个链表节点对应于 1 个数据块写操作,链表节点作为 CDP 元数据,它包括 2 个字段:数据块的块号和后向链接字段。

CDP 日志中的链表节点在日志元数据区中顺序存放。日志能容纳的节点数为 M 。链表节点的序

号为 $1 \sim (M-1)$,节点记为 $C_k (1 \leq k \leq M-1)$,CDP系统不使用链表节点0. 设置1个计数器,从1开始计数,每记录一个数据块写操作(即在日志中加入1个CDP元数据)后,计数器加1. 链表节点的序号就是当前时刻计数器的计数值.

对于第 k 个数据块写操作(计数器为 k 时接收到的写操作, $1 \leq k \leq M-1$),CDP系统把要写入该数据块的数据保存到CDP日志的数据区中(块地址为 k),把这个操作的元数据存储存储在链表节点 C_k 中. C_k 中存储的内容为: C_{k-L} (该数据块的块号 L)和 C_{k-r} 后向链接(该数据块的上一次写操作的元数据节点序号 r).

图1表示了15次数据块写操作的链表节点. 共有8个数据块,块号为 $0 \sim 7$,数据块5未被写入. 每个链表节点表示1次数据块写入操作,图中的横坐标是链表节点 $C_1 \sim C_{15}$ 的时刻,纵坐标是写入的数据块块号. 例如,数据块6在时刻7和时刻13被写入,生成 C_7 和 C_{13} . 图中实线箭头表示后向链接.

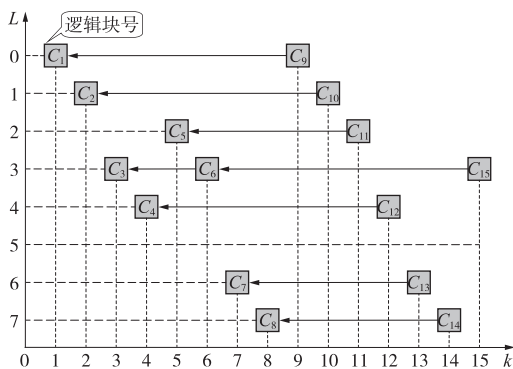


图1 后向链接数据结构

Fig.1 Back-link data structure

1.3 映射表

CDP系统在内存中设置1个映射表 S ,一共 N 项,每项记录当前时刻每个磁盘数据块的最后1次修改操作,即记录该操作的CDP元数据(链表节点)的序号. 节点序号为0时,表示该数据块未被修改. S 中的每项在系统初始化时被置为0.

1.4 临时映射表

在恢复某个时间点 k 的数据状态时,CDP系统在内存中建立1个临时映射表 T , T 一共有 N 项,在初始化时被置为特殊值 -1 ,提供一个磁盘映像供计算机访问. 计算机读取磁盘映像某个数据块时,CDP系统通过 T 获取该磁盘数据块在时间点 k 前的最后一次修改操作位置,即该操作的节点序号.

节点序号为0时,表示该数据块在时间点 k 时未被修改;节点序号为 -1 时,表示该数据块的位置尚未确定.

1.5 秒级索引

恢复时间点可以精确到秒级,CDP系统设置1个索引表,每秒在表的末尾增加1项,记录当前时刻计数器的计数值.

1.6 磁盘原始数据区

磁盘原始数据区 R 的容量和要保护的磁盘相同,容纳 N 个数据块. 在建立CDP系统时,将磁盘上的所有数据块复制到磁盘原始数据区.

2 快速恢复方法的相关算法

块级CDP系统的相关算法主要分为2个部分,即在系统执行连续数据保护任务时,被保护数据以及相关元数据的写入算法,及用户使用CDP系统对被保护磁盘进行任意时间点恢复时,生成历史磁盘镜像的相关算法. 由于快速恢复方法在被保护数据的元数据中加入了链接信息,所以需要对记录保护数据的算法进行相应的修改. 而在进行任意时间点数据恢复时,快速恢复方法使用按需恢复算法,从而提高了临时磁盘镜像的生产速度.

2.1 记录数据块写入操作

CDP系统维护1张映射表,记录磁盘中的所有数据块的最后1次修改操作. 对于每个写入的数据块,CDP系统生成1个链表节点. CDP系统在节点中记录数据块的块号,还要设置本节点的后向链接字段,以及映射表中相应块号的最后1次修改位置;数据块和链表节点分别写入CDP日志.

CDP系统中记录1个磁盘数据块写操作(块号为 $B, 0 \leq B \leq N-1$,要写入该块的数据为 D)的算法如下.

第1步 取得当前计数器的计数值为 k ,计数值加1; $k \geq M$ 时,日志区满,系统不再记录数据块写操作.

第2步 将原始数据 D 写入到日志的数据区 k .

第3步 构造此次数据块写操作的元数据即链表节点 C_k, C_k 的2个字段为: C_{k-L} 和 C_{k-r} .

① $L=B$,即记录该数据块对应的块号.

② $r=S[B]$,本链表节点 C_k 的后向链接指向该数据块的上一次修改位置,即上一次修改操作的链表节点在CDP元数据区中的序号.

第4步 更新映射表 $S, S[B]=k$.

2.2 按需快速恢复

当需要将 CDP 保护的磁盘恢复到之前某个指定时刻时,快速恢复方法分2步完成按需的磁盘内容恢复.首先需要在短时间内建立起一个临时磁盘映像,并提供给用户.在用户对临时磁盘映像发起读请求的过程中,逐步恢复磁盘地址和数据块间的映射关系(为简单起见,临时磁盘映像一般设置为只读状态).

2.2.1 建立磁盘映像

当需要恢复系统到某个指定时间点时,首先建立一个临时映射表,并将其中的内容全部设为-1.之后即可将磁盘映像作为一个逻辑单元(LUN)提供给计算机使用.

用户指定一个时间(单位为s),CDP系统执行以下操作,建立该时刻的磁盘映像供计算机访问.

① 在秒级索引 I 中获取该时刻的计数值,记为 k .

② 将临时映射表 T 中的每一项初始化为-1.

③ 使用 T 构造1个虚拟磁盘,作为1个逻辑单元(LUN)供计算机访问.

2.2.2 响应对磁盘映像的读取请求

在建立指定时间点 k 的磁盘映像后,CDP系统就可以接受计算机对磁盘映像的读取请求.当计算机对磁盘映像发起读请求时,CDP系统根据读请求中的数据块号,获得临时映射表的对应表项.当表项内容为-1时,从系统当前映射表开始通过后向链接向前查找,一直查找到指定时间点之前的最新节点,获取到该时间点这个数据块在 CDP 日志数据区中的位置(块号),并将它保存到临时映射表的对应表项中,之后根据临时映射表的表项内容,从日志数据区中读出数据块内容,返回给计算机.

从计算机的角度看,磁盘映像保存某一时刻所有磁盘数据块的内容,当收到一个读请求后,系统按顺序执行以下步骤:

① 从读请求中获取数据块号 B .

② 读取临时映射表 T 中数据块号 B 的表项内容,即 $T[B]$.

③ 如果 $T[B]=-1$,从当前映射表 S 中读取 $S[B]$,从节点 $C_{S[B]}$ 开始,沿着后向链接查找.若查找过程中某链表节点 C_i 的序号 $i \leq k$,则记录 $T[B]=i$,查找结束;如果链表中某个链表节点的后向链接 $C_{j-r}=0$,数据块 B 在时间点 k 之前没有被

修改,设置 $T[B]=0$,查找结束.

④ 如果 $T[B]=0$,则直接从磁盘原始数据读出数据块(块号为 B)返回给计算机;否则在 CDP 日志数据区中读出数据块(块号为 $T[B]$)返回给计算机.

例如在图1中,当前时刻为15,则当前系统映射表 S 的内容为

$$S[0]=9, S[1]=10, S[2]=11, S[3]=15, \\ S[4]=12, S[5]=0, S[6]=13, S[7]=14.$$

要建立指定时间点 $k=7$ 的磁盘映像,需要初始化1张临时映射表 T ,其中各项的值全部为-1.

若此时收到读请求,要读取块号为3($B=3$)的数据块.由于 $T[3]=-1, S[3]=15$,该数据块的后向链接节点为 C_{15}, C_6, C_3 .

查找过程中,沿该链表一直查找到 C_6 才满足条件($i=6, 6 \leq 7$),设置 $T[3]=6$,将 CDP 日志数据区中块号为6的数据块读出,返回给计算机.以后,如果计算机要读取数据块3时,由于 $T[3]=6$,直接从日志数据区中读出块号为6的数据块即可,而不需遍历后向链表.

若又收到读请求,要读取块号为7($B=7$)的数据块.该数据块的后向链接节点为 C_{14} 和 C_8 .查找过程中,没有满足条件($i \leq k$)的节点 C_i ,设置 $T[7]=0$,从磁盘原始数据区中读出数据块(块号为7),返回给计算机.

3 实验结果

为了说明使用了快速恢复方法的 CDP 系统的运行效率,使用实际工作负载数据对快速恢复方法进行了模拟实验.

实验使用在实际工作负载中捕获的数据,用于测试 CDP 快速恢复方法在典型工作负载下的表现情况.实验数据来自两个金融行业的在线事务处理系统.样本的基本数据总结在表1中.因为每个样本实际上包含了10个以上 LUN 的数据,本次只是从每个样本中取出了请求数量最多的 LUN 进行实验.

实验中,忽略 Trace 中的每一个可能的读请求,对于 Trace 中的每一个写请求,根据其中每个写入块更新映射表 S ,并且保存数据块对应的元数据结构(即链表节点),在处理完每一个写请求之后,计算在该时间点执行快速恢复后读取磁盘块需要搜索后向链接的平均次数.为了说明当恢复时间点距离当前时间点较近时的情况,分别计算恢复到当前时间

点之前 600,1 800,3 600 和 7 200 s 时,第一次读取临时映像的某磁盘块时所需后向搜索的平均迭代次数. 在计算量比较大的情况下,每隔固定个数的写请求执行 1 次计算.

表 1 样本特征

Tab. 1 Sample characteristic

样本	写请求个数	总请求个数	写请求所占比例/%	总写入块数	写入块号范围	最小写入块号	最大写入块号	块平均写入次数	块最大写入次数	平均写请求长度/数据块
样本 1	394 562	779 517	50.62	2 138 512	306 466	15	888 266	6.977 975	31 782	5.42
样本 2	131 868	943 276	13.98	312 390	50 978	107 426	1 192 069	6.127 938	13 642	2.37

将得到的数据总结在图 2 和图 3 中. 图中不同的曲线代表了不同的恢复时间点,实验开始经过时间小于恢复时间时即恢复到实验开始时间点.

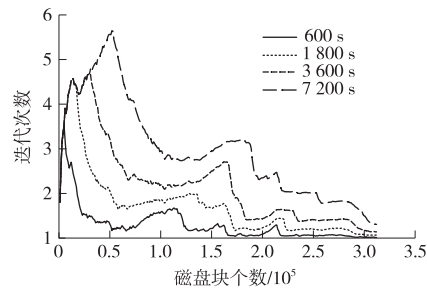


图 2 样本 1 后向连接平均迭代次数

Fig. 2 Average back-link search length of sample 1

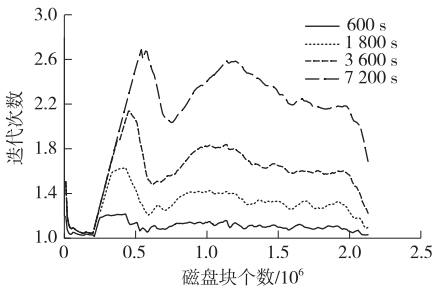


图 3 样本 2 后向连接平均迭代次数

Fig. 3 Average back-link search length of sample 2

显然,随着恢复时间点远离当前时间点,后向搜索迭代次数也必然单调增加,因此图中 1 800 s 的曲线始终在 600 s 曲线的上方,类似地,7 200 s 的曲线也在 3 600 s 的曲线上方. 考虑在极端情况下,即恢复到系统开始时间点的情况,可以从数据表中得知,在系统运行 12 h 后,恢复到系统开始时,平均后向搜索迭代次数仍然不到 7 次.

从结果图形中可以看出,虽然 2 个样本的数据在块平均写入次数上相差不多,都在 6 次和 7 次之间,但当恢复时间点距离当前时间点比较近时,2 个样本却体现出了较大差异. 样本 1 的写入频率可以看出是比较平均的,因为在系统开始不久后向搜索迭代次数增长到 2 后,除了一次回落,基本上保持在

3 以下. 这说明系统在 5×10^5 个写入块之后一段时间里,写入频率降低,平均后向搜索迭代次数也同时降至 2 以下,而在 1×10^6 个写入块之后,写入频率又恢复到了之前水平,从而后向搜索迭代次数也恢复到之前的 2 以上. 在样本 2 的结果中可以看出,在实验开始后不久,后向搜索迭代次数就迅速上升,恢复到 2 h 前的时间点甚至需要接近 6 次后向搜索. 可见系统开始时的平均写入频率比样本 1 高出很多,但之后的时间里后向搜索迭代次数不断减少,在系统运行 12 h 后几乎接近 1. 这说明样本 2 的系统只在开始运行时对磁盘进行了频繁的覆盖写入操作,之后的时间里写入频率逐渐降低. 因此可以知道,只要在单位时间里覆盖写入操作不是非常密集,使用快速恢复方法提供的磁盘映像是具有完全可以接受的性能的.

4 结 论

提出了一种连续数据保护系统的快速恢复方法. 针对连续数据保护系统恢复被保护磁盘时需要遍历 CDP 日志,从而需要较长时间的问题,提出了使用即时生成的磁盘映像,同时按需恢复磁盘数据的方法从而加快对当前时间点附近时间点恢复的速度.

快速恢复方法不需要遍历 CDP 日志即可提供磁盘映像. 当计算机读取磁盘映像的某个数据块时,CDP 系统根据链表从 CDP 元数据中获取该数据块的映射关系,实现了磁盘数据块的按需恢复. 适合于用户需要快速建立磁盘映像,读取其中一小部分文件的情况. 另外快速恢复方法不依赖快照即可进行数据快速恢复. 通过在 CDP 元数据中加入链接信息,使得可以从当前时间点快速回溯到指定的时间点,而不必从 CDP 日志的开始时间点或快照时间点开始遍历,特别是对距离当前时间点较近的时间点,具有较好的执行效率.

参考文献:

- [1] Wallis J. Common causes for data loss [EB/OL]. [2008-10-08]. <http://www.isnare.com/?aid=222505&ca=Computers+and+Technology>.
- [2] Xiao Weijun, Ren Jin, Yang Qing. Case for continuous data protection at block level in disk array storages[J]. IEEE Transactions on Parallel and Distributed Systems, 2009,20(6):898-911.
- [3] Li Xu, Xie Changsheng, Yang Qing. Optimal implementation of continuous data protection (CDP) in Linux Kernel [C] // Proceedings of International Conference on Networking, Architecture, and Storage. Chongqing, China: IEEE Computer Society, 2008: 28-35.
- [4] Zhu Ningning, Chiueh Tzicker. Portable and efficient continuous data protection for network file servers[C]// Proceedings of 37th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN'07). Edinburgh: IEEE Computer Society, 2007: 687-697.
- [5] Lu Maohua, Chiueh Tzicker, Lin Shibiao. An incremental file system consistency checker for block-level CDP systems [C] // Symposium on Reliable Distributed Systems. Naples: IEEE Computer Society, 2008:157-162.
- [6] Lu Maohua, Chiueh Tzicker. File versioning for block-level continuous data protection [C] // Proceedings of 2009 29th IEEE International Conference on Distributed Computing Systems. Montreal: IEEE Computer Society, 2009:327-334.
- [7] Sheng Yonghong, Wang Dongsheng, He Jinyang, et al. TH-CDP: an efficient block level continuous data protection system [C] // Proceedings of 2009 IEEE International Conference on Networking, Architecture, and Storage. Zhangjiajie, China: IEEE Computer Society, 2009:395-404.
- [8] Lu Maohua, Lin Shibiao, Chiueh Tzicker. Efficient logging and replication techniques for comprehensive data protection [C] // Proceedings of 24th IEEE Conference on Mass Storage Systems and Technologies (MSST 2007). San Diego, USA: IEEE Computer Society, 2007:171-184.
- [9] Brinkmann A, Effert S. Snapshots and continuous data replication in cluster storage environments [C] // Proceedings of 4th International Workshop on Storage Network Architecture and Parallel I/Os. San Diego, USA: IEEE Computer Society, 2008:3-10.
- [10] Yang Qing, Xiao Weijun, Ren Jin. TRAP-array: a disk array architecture providing timely recovery to any point-in-time [C] // Proceedings of the 33rd International Symposium on Computer Architecture (ISCA'06). Boston, USA: IEEE Computer Society, 2006:289-301.
- [11] Liu Jingning, Yang Tianming, Li Zuoheng, et al. TSPSCDP: a time-stamp continuous data protection approach based on pipeline strategy [C] // Proceedings of Japan-China Joint Workshop on Frontier of Computer Science and Technology. Nagasaki: IEEE Computer Society, 2008:96-102.

(责任编辑:刘芳)